Big Data
Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria
Academic year 2026/2027

Search Engine Project

Stage 2: Building a Service-Oriented Architecture

Group: **UAV-2**

Full name	**Student ID**
Hugo Amador Hidalgo	54173521B
Jesús Varela Vasquez	Y9772562Z
Amai Suárez Navarro	45780500N

Repository: `https://github.com/UAV-2/Stage_2`

Contents

1 Introduction and objectives

2 System architecture

2.1 Services and deployment

2.2 Coordination mechanism

2.3 Processes, replicas and threads

2.4 Shared storage

3 API design and contracts

3.1 Front-end API

3.2 Internal service APIs

3.3 Operational and administrative endpoints

3.4 Non-HTTP contracts

3.5 Specification and validation evidence

4 Microservice implementation

4.1 Code structure

4.2 Ingestion Service

4.3 Indexing Service

4.4 Search Service

4.5 Control Module

5 Design decisions

5.1 Coordination

5.2 Single-machine deployment

5.3 API design

5.4 Storage

5.5 Concurrency model, replication and threading

6 Benchmarks and results

6.1 Experimental setup

6.2 E1: ingestion and indexing pipeline

6.3 E2: search under concurrent load

6.4 CPU and memory

6.5 JMH microbenchmarks

6.6 Bottlenecks and limitations

7 Conclusions and future improvements

7.1 Conclusions

7.2 Evolution towards the Stage 3 cluster

1 Introduction and objectives

Stage 1 built the data layer of the search engine as a single program: a *datalake* with the raw Project Gutenberg books split into header and body, two *datamarts* (a metadata table and an inverted index) and append-only control files. Stage 2 turns that data layer into a **service-oriented architecture**: independent processes with REST APIs that cooperate through an explicit coordination mechanism. All processes, including the replicas of each service, run on one machine; the deployment on a cluster belongs to Stage 3.

The objectives achieved in this stage are:

1. **Three cooperating services** written in Java 17 with Javalin and Gson (Ingestion, Indexing and Search), each built by Maven as an executable `.jar` and configured through environment variables.
2. **Explicit coordination.** A *Control Module* orchestrates the workflow over HTTP (ingest → index → publish), records progress persistently and retries failed work. The visibility of data between services is governed by documented shared-storage contracts: a readiness marker for ingested books and an atomically published index version for search.
3. **A documented API.** The front-end search API is separated from the internal and operational endpoints. It is specified in OpenAPI 3, listed in an endpoint inventory, and validated by runnable examples and an automatic check script (54 of 54 checks pass).
4. **Replicas and threads.** Every service can be started with several replicas and a configurable number of HTTP threads; concurrent writers are coordinated with file locks and atomic renames.
5. **Benchmarks.** JMH microbenchmarks of the core operations and integration benchmarks of the whole system with real Gutenberg books, which identify the first bottleneck (publication of the index) and the changes needed for Stage 3.

2 System architecture

2.1 Services and deployment

The system is a Maven multi-module project. Figure 1 shows the processes, how they communicate and the shared storage. Every process runs on the same host and listens on `localhost`; the ports, data directory, replica counts and thread counts are configurable.

- **Ingestion Service** (port 7001). Downloads a book from Project Gutenberg (or reads it from a local folder in the benchmarks), splits header and body and stores both files in the datalake.
- **Indexing Service** (port 7002). Reads an ingested book, extracts its metadata into SQLite and tokenizes the body. The term frequencies stay pending in memory until a *publish* call writes them as a new immutable index segment and publishes a new index version.
- **Search Service** (port 7003). Serves the front-end API: keyword search (AND semantics, tf-idf ranking) with author, language and year filters and pagination, book details and filter values. It only reads published index versions.
- **Control Module** (port 7000). The orchestrator: it decides which book is processed, calls the internal APIs, records the progress and retries.

Replica r of a service listens on its base port + $10 \cdot r$ (for example, the second search replica uses 7013). The script `scripts/start_all.sh` starts the whole deployment with the variables `INGESTION_REPLICAS`, `INDEXING_REPLICAS`, `SEARCH_REPLICAS` and `HTTP_THREADS`.

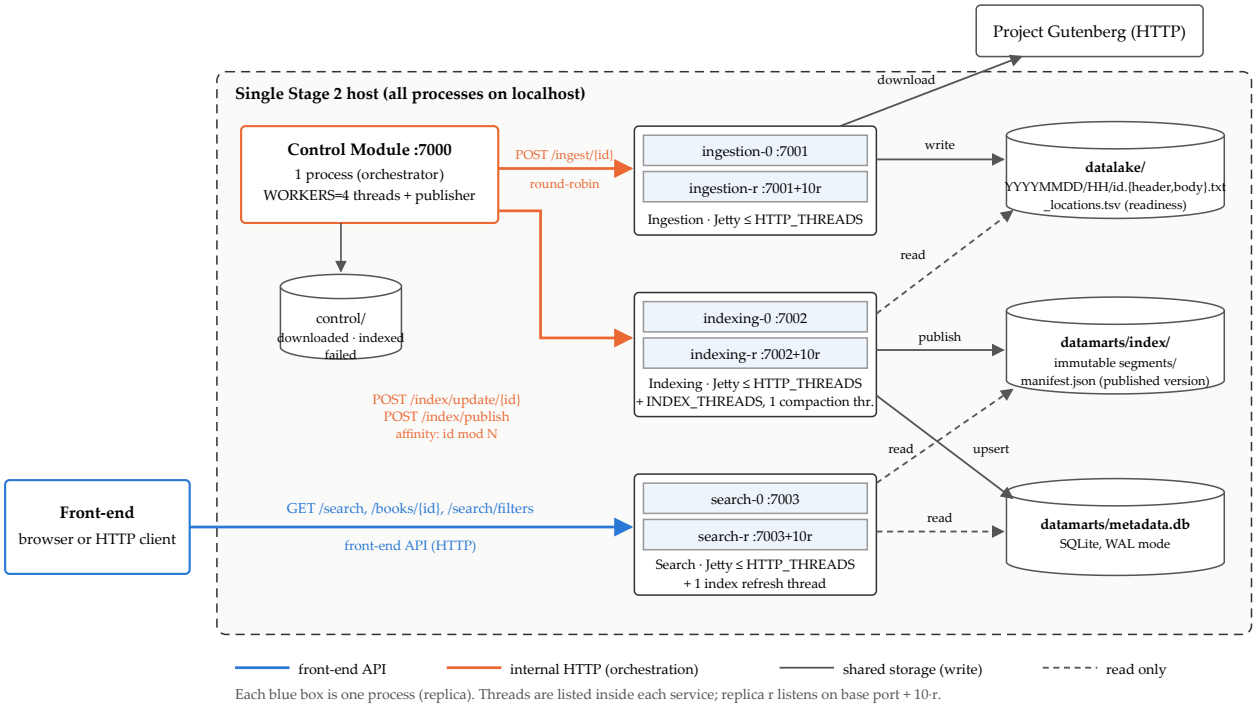


Figure 1. Architecture. Every process runs on one host. The front-end only talks to the Search Service; the Control Module drives Ingestion and Indexing through internal HTTP calls; data between services flows through the shared storage, under the contracts of Section 3.4.

2.2 Coordination mechanism

We use **classical orchestration** combined with **shared-storage contracts**. The Control Module owns the workflow: it is the only component that decides which book is processed next, it calls the services, waits for their answer and records each completed step in its control files. The services do not call each other. They exchange data through the datalake and the datamarts, and the rules that make that exchange safe (when a book is ready, when an index version is visible) are part of the contracts in Section 3.4. Table 1 lists the steps for one book.

Table 1. Workflow for one book. `WORKERS` books are processed in parallel.

Step	Call (caller → provider)	Effect	Recorded by the Control Module
1	Control → Ingestion: `POST /ingest/{id}` (round-robin among replicas, next replica on failure)	Header and body written (temporary file + atomic rename), then the id is appended to `_locations.tsv`	200 → `downloaded_books.txt`; 404/422 → `failed_books.txt`
2	Control → Indexing replica *id* mod *N*: `POST /index/update/{id}`	Metadata upserted in SQLite; term frequencies kept pending in memory. 409 if the book is not ready	Book waits for publication
3	Control → same replica: `POST /index/publish`, every `PUBLISH_INTERVAL_MS` (2 s) or after `PUBLISH_BATCH` (25) pending books	New immutable segment, then `manifest.json` replaced atomically (new version)	200 → `indexed_books.txt` (book is searchable)
4	Search replicas (no call)	Re-read the manifest every `REFRESH_MS` (500 ms) and switch to the new version	–

A book becomes eligible for indexing only after ingestion has finished (step 1 writes the readiness line last), search always reads one complete published version (step 3 replaces the manifest atomically), and every step is idempotent, so repeating a request after a failure never creates duplicates.

2.3 Processes, replicas and threads

A service is a logical responsibility; its deployment has one or more *process replicas*, and every process uses several *threads*. Table 2 shows both levels for each component, how work is distributed and how the shared state is protected.

Table 2. Replicas, threads and protection of shared state. Defaults in parentheses.

Component	Replicas	Threads per process	Distribution of work	Shared state and protection
Control Module	1	`WORKERS` (4) book workers, 1 publisher thread, Jetty pool of the admin API	Reads `BOOK_IDS` in order	Control files: `synchronized` methods, file synced on every append
Ingestion	`INGESTION_REPLICAS` (1)	Jetty pool, max `HTTP_THREADS` (32)	Round-robin from the Control Module; on failure, next replica	Datalake: atomic rename; per-book lock (64 lock stripes) and lock of `_locations.tsv`, both `FileLock` (processes) + `ReentrantLock` (threads)
Indexing	`INDEXING_REPLICAS` (1)	Jetty pool (`HTTP_THREADS`); `INDEX_THREADS` (CPU count) for rebuilds; 1 compaction thread	Affinity *id* mod *N*: a book always goes to the same replica	Pending buffer (`ConcurrentHashMap`); in-process publication lock; `FileLock` on the manifest between replicas; SQLite in WAL mode with 10 s `busy_timeout`
Search	`SEARCH_REPLICAS` (1)	Jetty pool (`HTTP_THREADS`); 1 refresh thread	The client or a load balancer spreads queries	Read only: immutable snapshot per query (`AtomicReference`), synchronized LRU cache of posting lists (`CACHE_ENTRIES`, 50 000), one SQLite connection per thread

HTTP threads. Javalin 6 runs on Jetty. `HTTP_THREADS` sets the maximum size of Jetty's `QueuedThreadPool` (minimum `min(4, HTTP_THREADS)`); we always set it explicitly, so the framework default is not used. Virtual threads are disabled, which is the Javalin 6 default. When all threads are busy, Jetty queues the requests instead of rejecting them, and long operations (index rebuild) are administrative only, so request threads cannot be exhausted by the normal workflow. An in-process lock only protects the threads of that process: every lock that must also coordinate replicas is a file lock.

2.4 Shared storage

All data lives under `DATA_DIR` (default `data/`):

- `datalake/YYYYMMDD/HH/<id>.header.txt` and `.body.txt`: the time-based ingestion hierarchy of the assignment, implemented in Stage 1. `datalake/_locations.tsv` maps every complete book to its folder, so a book is located in one read instead of probing hour folders (the weakness of this layout measured in Stage 1), and the same file is the readiness marker.
- `datamarts/index/segments/<segment>/<LETTER>/<term>.txt`: immutable index segments that reuse the hierarchical layout of Stage 1 (lines `book_id \t tf`), and `datamarts/index/manifest.json` with the published version, its segments, its books and the retired segments.
- `datamarts/metadata.db`: SQLite table `books` (title, author, language, year, release date, body path), with indexes on author, language and year.
- `control/`: `downloaded_books.txt`, `indexed_books.txt` and `failed_books.txt`, written only by the Control Module.

3 API design and contracts

The full specification is `api/openapi.yaml` (OpenAPI 3, version 1.0.0), with every parameter, schema, status code and example. `api/ENDPOINTS.md` contains the inventory summarised here. All errors use the same JSON body `{"error", "message", "status"}`, including unknown routes (404 `NOT_FOUND`).

3.1 Front-end API

Provider: Search Service. **Consumer:** the front-end, or any HTTP client that exercises the same contract. The front-end use cases are: (1) search books by keywords, optionally filtered by author, language and year, page by page; (2) open the details of a result; (3) fill the filter controls with the available values. All three are read-only and synchronous (Table 3).

Table 3. Front-end endpoints.

Method	Route	Purpose	Status codes
GET	`/search?q&author&language&year&offset&limit`	AND keyword search with combinable filters and pagination	200, 400
GET	`/books/{book_id}`	Metadata of a searchable book	200, 400, 404
GET	`/search/filters`	Languages (with book counts), year range and top authors	200

Semantics of `/search`.

- *Parameters.* `q` is required (at most 500 characters). `limit` defaults to 20 (1–100) and `offset` to 0 (≥ 0).
- *Query.* `q` is tokenized exactly like the books and its terms are combined with AND. A query made only of stopwords returns 200 with `total: 0`.
- *Filters.* `author` is a case-insensitive substring; `language` is an exact code (`en`, `fr`…), case-insensitive; `year` is an exact year (original publication if the header gives it, otherwise the Gutenberg release year). Filters can be combined.

- *Ordering.* tf-idf score, highest first, with score = Σ (1 + ln tf)·ln(1 + N/df) over the query terms; ties by ascending `book_id`.
- *Pagination.* `total` counts every match and `results` contains only the requested page. An offset beyond `total` returns 200 with an empty list.
- *Errors (400).* `MISSING_QUERY`, `QUERY_TOO_LONG`, `INVALID_AUTHOR`, `INVALID_LANGUAGE`, `INVALID_YEAR`, `INVALID_OFFSET`, `INVALID_LIMIT`.
- *Consistency.* Each response carries the `index_version` it was computed from.
- *CORS.* Allowed origins are set with `CORS_ORIGINS` (default `*`), so a browser front-end served from another origin can call the Search Service directly.

```
GET /search?q=whale&language=en&offset=0&limit=20          (shape of the answer; values abbreviated)
200 OK
{"query": "whale", "terms": ["whale"], "filters": {"language": "en"},
 "total": <all matches>, "offset": 0, "limit": 20, "index_version": <published version>,
 "results": [{"book_id": ..., "title": ..., "author": ..., "language": "en", "year": ..., "score": ...}, ...]}
```

3.2 Internal service APIs

Consumer: Control Module. All internal calls are **synchronous**: a 200 means the work has been completed, not only accepted (Table 4).

Table 4. Internal endpoints. Step refers to Table 1.

Provider	Method and route	Purpose	Step	Idempotency and retries
Ingestion	`POST /ingest/{book_id}`	Download, split and store the book	1	Idempotent (`already_present`). Retry on network errors and 502; never on 404 (not in Gutenberg) or 422 (no START/END markers)
Ingestion	`GET /ingest/status/{book_id}`	Is the book available?	diag.	Read only
Ingestion	`GET /ingest/list`	Books in the datalake	diag.	Read only
Indexing	`POST /index/update/{book_id}`	Store metadata and keep the terms pending	2	Idempotent (`already_indexed`). 409 `BOOK_NOT_INGESTED` if not ready
Indexing	`POST /index/publish`	Make the pending books searchable (new version)	3	Idempotent: already published books are returned as `skipped`; without pending books nothing changes

Timeouts and failures for the caller. Every internal call has a timeout of `REQUEST_TIMEOUT_MS` (120 s). Network errors, timeouts and 5xx answers are retried up to `MAX_RETRIES` (3) times with exponential backoff from `RETRY_BACKOFF_MS` (500 ms); ingestion retries on the next replica. If an indexing replica fails to publish several times in a row, its pending books are sent to another replica. Whatever is not completed stays visible in the control files and is resumed by the next run.

3.3 Operational and administrative endpoints

Table 5. Operational and administrative endpoints (not used by the front-end).

Provider	Method and route	Consumer	Purpose
All	`GET /status`	Control Module (start-up wait), benchmark scripts, operators	Health, pid, replica id, threads, heap, process CPU and service statistics
Indexing	`GET /index/status`	Operators, benchmarks	Books indexed, version, segments, pending books, size
Indexing	`POST /index/rebuild`	Administration	Rebuild index and metadata from the datalake (synchronous)
Indexing	`POST /index/compact`	Administration	Merge segments (also automatic above `COMPACT_THRESHOLD` = 16)
Control	`POST /control/run?books=N`	Administration	Start a run; asynchronous: 202, or 409 if a run is in progress
Control	`GET /control/status`	Administration	Phase and metrics of the last run (how the caller observes completion)

3.4 Non-HTTP contracts

Part of the coordination happens through shared files on the single host. Each contract has one owner, defined readers and an explicit synchronization rule.

- **Datalake readiness.** Owner: Ingestion. Reader: Indexing. Body and header are written to temporary files and renamed atomically; only then is the line `book_id \t YYYYMMDD/HH` appended to `_locations.tsv`. A book is available *if and only if* its line exists.
- **Published index.** Owners: all Indexing replicas. Reader: Search. A segment is written to `<segment>.tmp` and renamed (no lock needed: names are unique). Then, holding `.manifest.lock`, the replica reads the manifest, adds the segment and its

books, and replaces the manifest atomically. Search re-reads it every `REFRESH_MS` and each query uses one version. Segments replaced by compaction or rebuild are deleted after `RETIRED_GRACE_MS` (60 s), far longer than any query. A book already listed in the manifest is never published again.

- **Metadata.** Owner: Indexing. Reader: Search. SQLite in WAL mode (one writer, many readers, also across processes), `busy_timeout` of 10 s and upsert by `book_id`. Metadata is written *before* the terms are published, and search always starts from the published index, so it never returns a book without metadata.
- **Workflow state.** Owner: Control Module (single process). A line is appended to a control file only *after* the service has confirmed the operation.

Table 6. What happens if the Control Module stops between an operation and its record.

Interruption	On restart
After ingestion, before recording it	Ingestion is repeated and answers `already_present: true`
After the update, before publication	If the replica is alive, the next publish includes the book; if it restarted, the book is in *downloaded − indexed* and is indexed again
After publication, before recording it	The update answers `already_indexed: true` and the book is recorded

3.5 Specification and validation evidence

- `api/openapi.yaml`: OpenAPI 3 specification of every implemented endpoint; `api/ENDPOINTS.md`: inventory separated into front-end, internal and operational endpoints, plus the non-HTTP contracts.
- `api/examples.http`: runnable requests (IntelliJ or VS Code REST client) with the expected answers.
- `scripts/check_api.py`: automatic checks against the running services. They cover ingestion (new and repeated book, status, list, invalid id), indexing (update, publish, repeated update, 409 for a book not ingested, status), book details (found, 404, 400), keyword search, each filter alone and combined, empty results, every 400 error code, pagination (`limit=1`, `offset=total`), filter values and unknown routes. Result on the benchmark machine, with real books: **54/54 checks passed**.
- 36 JUnit tests (`mvn test`), including API tests that start each service in-process and call it over HTTP, and concurrency tests of the datalake, the index and SQLite with several threads and instances.

4 Microservice implementation

4.1 Code structure

Each service is a Maven module that produces a self-contained executable jar (Maven Shade). The shared code is in the `common` library, so every service applies exactly the same processing rules.

Table 7. Maven modules. Libraries: Javalin 6.4.0, Gson 2.12.1, sqlite-jdbc 3.49.1, SLF4J 2.0.16, JUnit 5.11, JMH 1.37.

Module	Main classes	Role
`common`	`GutenbergText`, `HeaderParser`, `Tokenizer`, `Datalake`, `FileLocks`, `AtomicFiles`, `MetadataStore`, `InvertedIndex`, `Segment`, `Manifest`, `IndexReader`, `ServiceServer`, `ServiceClient`, `Config`	Text processing, datalake, metadata, segmented index, common Javalin server (JSON, uniform errors, CORS, thread pool, `/status`) and HTTP client
`ingestion-service`	`IngestionApp`, `IngestionService`, `BookSource`	Routes `/ingest/*`; Gutenberg or local source
`indexing-service`	`IndexingApp`, `IndexingService`	Routes `/index/*`; pending buffer, publication, rebuild, compaction
`search-service`	`SearchApp`, `SearchEngine`, `SearchQuery`	Front-end API, ranking, filters and pagination
`control-module`	`ControlApp`, `Orchestrator`, `ControlState`, `RunMetrics`	Orchestration, control files, retries, metrics, admin API
`benchmarks`	`TextProcessingBenchmark`, `IndexPublishBenchmark`, `SearchBenchmark` (in `src/test/java`)	JMH microbenchmarks, excluded from the service jars

4.2 Ingestion Service

`POST /ingest/{id}` first takes the per-book lock and checks `_locations.tsv`; if the book is already there it answers `already_present` without downloading it again. Otherwise it fetches the text (`BookSource`: Gutenberg over HTTP with a politeness delay of `SOURCE_DELAY_MS` = 1 s between requests of the same process, or a local folder), locates the START/END markers, writes body and header atomically and finally appends the readiness line. Missing books return 404, files without markers 422 and download errors 502.

4.3 Indexing Service

`POST /index/update/{id}` resolves the book through `_locations.tsv` (409 if it is not ready), parses the header (title, author, language, year and release date), upserts the metadata row and tokenizes the body into term frequencies, which are kept in a pending buffer. `POST /index/publish` takes the in-process publication lock, writes all pending books as one new segment and updates the manifest under the file lock; it answers with the new version and the lists of `published` and `skipped` books. When the number of segments exceeds `COMPACT_THRESHOLD`, a background thread merges them into one segment and retires the old ones. `POST /index/rebuild` rebuilds everything from the datalake with `INDEX_THREADS` threads.

4.4 Search Service

A refresh thread loads the manifest every 500 ms and publishes an immutable snapshot through an `AtomicReference`; each query takes the current snapshot once, so it never mixes two versions. For each query term the engine reads the posting lists of all segments of the snapshot (through an LRU cache, valid because segments are immutable), intersects them, computes the tf-idf score, applies the filters with the metadata in SQLite and returns the requested page. Every request thread has its own SQLite connection, because JDBC connections are not thread-safe.

4.5 Control Module

The orchestrator runs `WORKERS` books in parallel and a publisher thread that calls `/index/publish` on each indexing replica every 2 s or when 25 books are pending there. Before starting, it waits for the `/status` of every service and loads its control files, so a new run skips what is already done and resumes the rest. It also measures the duration of every call and the time each book waits until it is searchable (Section 6.2). The admin API (`/control/run`, `/control/status`) starts runs asynchronously and reports their progress.

5 Design decisions

5.1 Coordination

- **Why orchestration.** The workflow is a strict sequence per book (ingest, then index, then publish), and the retry policy, the record of progress and the metrics are easier to get right in one place. It needs no extra process such as a broker, and every step can be observed and repeated by hand with the same HTTP calls.
- **Why an explicit publish step.** Publishing one book at a time would create one segment per book. Separating *update* (cheap, per book) from *publish* (expensive, per batch) lets the Control Module choose the batch size, which Section 6 shows is the dominant cost.
- **Why shared-storage contracts.** On one host the services already share the file system; documented readiness markers and atomic publication give the guarantees the assignment requires (no indexing before ingestion, consistent published state, no duplicates) without sending book contents over HTTP.
- **Cost.** The Control Module is a single point of failure. It keeps no state in memory that cannot be rebuilt from its control files, so a restart resumes the work; in Stage 3 it should be replaced by a persistent queue or by leader election.

5.2 Single-machine deployment

Every setting is an environment variable: ports (base + $10 \cdot r$ for replicas), `DATA_DIR`, source, replica counts, thread counts, batch sizes, timeouts and retries. `start_all.sh` checks that the ports are free, starts every replica, waits for its `/status` and records its pid; `stop_all.sh` stops them all. The same scripts are used by the benchmarks, so every experiment can be reproduced with one command.

5.3 API design

Search is one endpoint with optional filters instead of one route per filter, so the front-end builds a single URL. Errors share one JSON format with stable codes, so a client can react without parsing text. Internal calls are synchronous because each one finishes in milliseconds (except *publish*), which makes completion trivial to observe; the only long-running operation started through the API (`/control/run`) is asynchronous and has a status endpoint. Responses expose `index_version` so that a client can detect a change of version between pages.

5.4 Storage

- **Datalake.** The assignment asks to preserve the ingestion hierarchy (`YYYYMMDD/HH`). We keep it and add the `_locations.tsv` catalogue, which removes the slow lookup measured in Stage 1 and doubles as the readiness marker.

- **Index.** Immutable segments plus a manifest give snapshot isolation without readers ever taking a lock: writers only add files, and the manifest swap is the single atomic commit point. Segments reuse the Stage 1 hierarchical layout (one file per term), which was the fastest layout for queries. Section 6 shows that its weak point, creating many small files, is now the main bottleneck.
- **Metadata.** SQLite was the fastest backend in Stage 1 and needs no server. WAL mode lets the search replicas read while an indexer writes, and `busy_timeout` plus retries when the database is opened handle several replicas starting at the same time.

5.5 Concurrency model, replication and threading

- **Duplicate work.** Ingestion takes a per-book lock and checks the catalogue; indexing affinity sends a book always to the same replica; the manifest records published books, so a repeated publish skips them.
- **Conflicting writes.** Segment names are unique, so only the manifest needs mutual exclusion, and it is held only for a read-modify-write of one small file. SQLite serialises writers by itself.
- **Thread exhaustion.** Bounded Jetty pools with a queue; only administrative calls are long.
- **Where replication or threads help.** Search is read-only and its queries are independent, so both threads and replicas can add throughput until the CPU is full (Section 6.5 measures 6.3× with 8 threads). Ingestion from Gutenberg is limited by the politeness delay of each process, so ingestion replicas would increase the download rate.
- **Where they add contention.** All indexing replicas publish to the same manifest and the same disk; more replicas also produce more, smaller segments in which frequent terms are repeated. Section 6.2 measures this.

6 Benchmarks and results

6.1 Experimental setup

Table 8. Host and protocol. All processes, including the load generator, run on this machine.

CPU	Intel Core i7-14700K: 20 cores (8 performance + 12 efficiency), 28 logical CPUs
Memory / storage	32 GB RAM; NVMe SSD (WD PC SN560, 1 TB), NTFS
Software	Windows 11 Home, Git Bash; Java 21.0.7 (code compiled for Java 17); JMH 1.37; Python 3 for the load generator and the monitor
Data	Real Project Gutenberg books downloaded once into a local cache (`scripts/fetch_books.py`). The integration runs use 100 of them through `SOURCE=local:...`, so network time is excluded
E1 pipeline	Full run of 100 books (ingest + index + publish) with fresh data; 5 configurations of `WORKERS` and `INDEXING_REPLICAS`, 3 repetitions each
E2 search	Index of 100 books; closed-loop clients (each one sends the next query as soon as it gets the answer) at 1–32 concurrent clients; 15 s per point after a warm-up, 3 repetitions; 11 query templates with filters and pagination; 4 configurations of `SEARCH_REPLICAS` and `HTTP_THREADS`
Resources	CPU and resident memory of every process and of the host, sampled every 0.5 s (`scripts/monitor.py`, psutil)
JMH	5 warm-up + 10 measured iterations, 2 forks; heap fixed at 2 GB in the main suite; threading experiment with `-t 1, 2, 4, 8`

Client concurrency (E2), internal worker threads (`WORKERS`, `HTTP_THREADS`, JMH `-t`) and process replicas are reported separately. All numbers are means ± standard deviation of the repetitions unless stated otherwise. The whole experiment is reproduced with `scripts/integration_bench.sh`, `scripts/run_jmh.sh all`, `scripts/run_jmh.sh threads` and `benchmarks/plot_results.py`.

6.2 E1: ingestion and indexing pipeline

Table 9. Pipeline with 100 books. Call latencies are mean / p95 in ms; publish wait is the time from a book's update to the confirmation that it is searchable (mean / p95, in s).

Workers	Indexing replicas	Total time (s)	Books/s	Ingest call (ms)	Update call (ms)	Publish call (s)	Publish wait (s)	Retries / failed
1	1	58.8 ± 8.9	1.73 ± 0.24	11.1 / 27	9.4 / 25	16.1	22.3 / 27.8	0 / 0
4	1	44.3 ± 1.2	2.26 ± 0.06	11.2 / 27	11.7 / 36	21.9	37.7 / 43.8	0 / 0
8	1	42.3 ± 0.8	**2.36 ± 0.05**	15.0 / 35	18.3 / 69	21.0	35.2 / 42.0	0 / 0
8	2	51.4 ± 3.0	1.95 ± 0.11	16.0 / 39	18.3 / 67	18.8	30.9 / 50.9	0 / 0
8	4	52.2 ± 19.8	2.10 ± 0.72	66.3 / 168	25.5 / 88	24.3	37.1 / 47.0	0 / 0

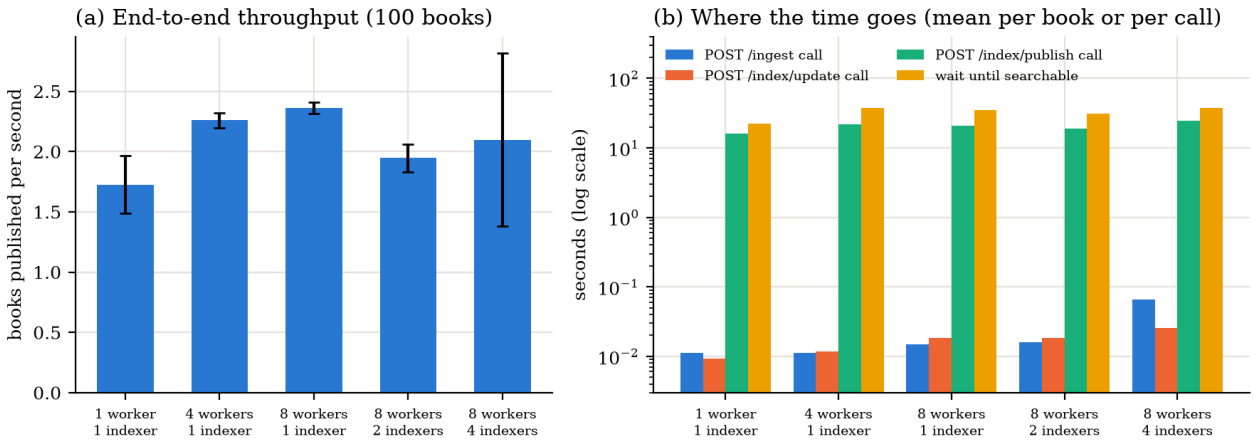


Figure 2. E1 pipeline. (a) Books published per second (mean ± sd of 3 runs). (b) Mean duration of each internal call and mean time a book waits until it is searchable, on a logarithmic scale.

- **Correctness.** All 15 runs published the 100 books with no retry and no failure.
- **Workers.** Going from 1 to 4 workers improves throughput by 31 % (1.73 → 2.26 books/s); from 4 to 8 only by 4 %. Ingestion and update calls take 10–20 ms, so more parallel workers stop helping as soon as books queue for publication.
- **The bottleneck is publication.** A publish call takes 16–24 s on average, three orders of magnitude more than the other calls (Figure 2b), and a book waits 22–38 s on average until it is searchable. The total time of a run (42–59 s) is essentially a few consecutive publications. This is the coordination cost of our design: the orchestrator itself adds milliseconds, but the step that makes data consistently visible dominates.
- **Why publication is slow.** Each segment stores one file per term. The 100-book index left by the last run has 7 segments and **257 208 term files** (between 9 557 and 74 270 per segment). Creating tens of thousands of small files is slow on NTFS; the indexing process averages about one core while the whole host stays at 11–24 % CPU (Section 6.4), so the limit is file-system work, not computation. JMH confirms it in isolation (Section 6.5).
- **Indexing replicas do not help on one host** (1.95 and 2.10 books/s with 2 and 4 replicas against 2.36 with one). Every replica writes its own segments to the same disk and takes the same manifest lock, and splitting the books among replicas repeats the frequent terms in more segments, so more files are created in total. With 4 replicas the variance also grows a lot (36–74 s), with ingestion latencies up to 395 ms (p95) in the slowest run: contention for the disk.

6.3 E2: search under concurrent load

Table 10. GET /search with an index of 100 books (mean ± sd of 3 runs; 72 measurement points, 0 errors in all of them).

Configuration	1 client (q/s)	4 clients	8 clients	32 clients	p95 @1 (ms)	p95 @8	p95 @32
1 replica, 8 HTTP threads	3 476 ± 59	10 960 ± 211	11 262 ± 473	10 930 ± 313	0.44	1.29	6.45
1 replica, 32 HTTP threads	3 392 ± 143	12 162 ± 78	12 248 ± 270	11 868 ± 151	0.48	1.18	5.77
2 replicas, 32 HTTP threads	3 968 ± 72	11 798 ± 372	12 004 ± 247	10 789 ± 256	0.39	1.20	6.58
4 replicas, 32 HTTP threads	3 899 ± 193	11 229 ± 272	11 748 ± 303	11 567 ± 141	0.40	1.22	5.98

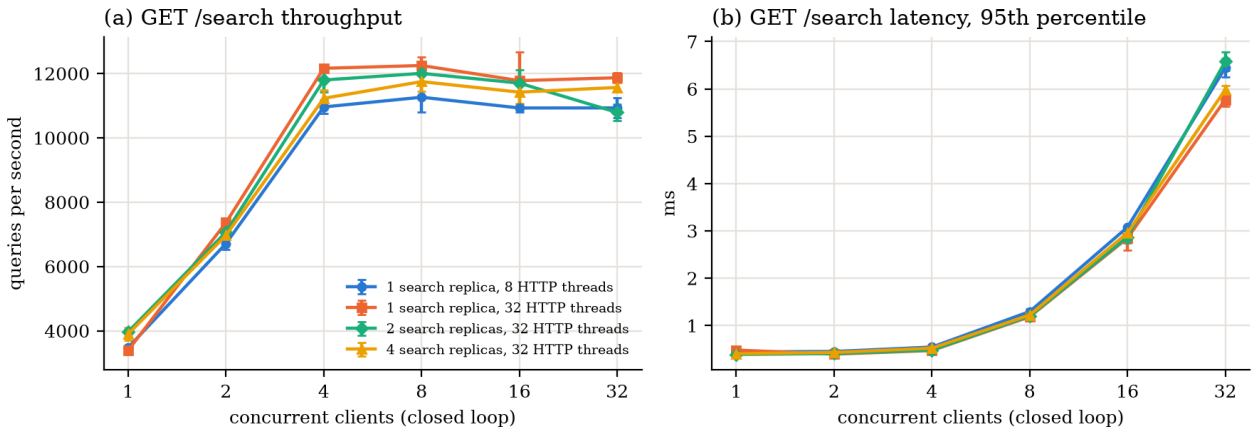


Figure 3. E2 search. (a) Throughput and (b) 95th-percentile latency against the number of concurrent clients, for each replica and thread configuration (mean ± sd of 3 runs).

- **Saturation.** Throughput grows almost linearly up to 4 clients and then stays at 11 000–12 000 queries/s. Beyond that point more clients only add queueing: the p95 latency goes from 0.4–0.5 ms (1 client) to 1.2–1.3 ms (8) and 5.8–6.6 ms (32). The mean latency follows Little's law, $R = N/X$: with 32 clients and 11 868 q/s it predicts 2.70 ms, and 2.69 ms was measured.
- **Replicas and HTTP threads have almost no effect** (differences within about 10 %). The reason is that the search processes are not the limit: all search replicas together use about 2.4 cores in every configuration and the host never exceeds about 5 of its 28 logical CPUs (Section 6.4). The limit is the load generator, a Python process on the same machine. This is a limitation of our evidence, not a property of the service: measuring the benefit of search replicas requires a faster client or a client on another machine.
- **Errors.** None in the 72 points, including the configurations with 4 replicas and 32 clients.

6.4 CPU and memory

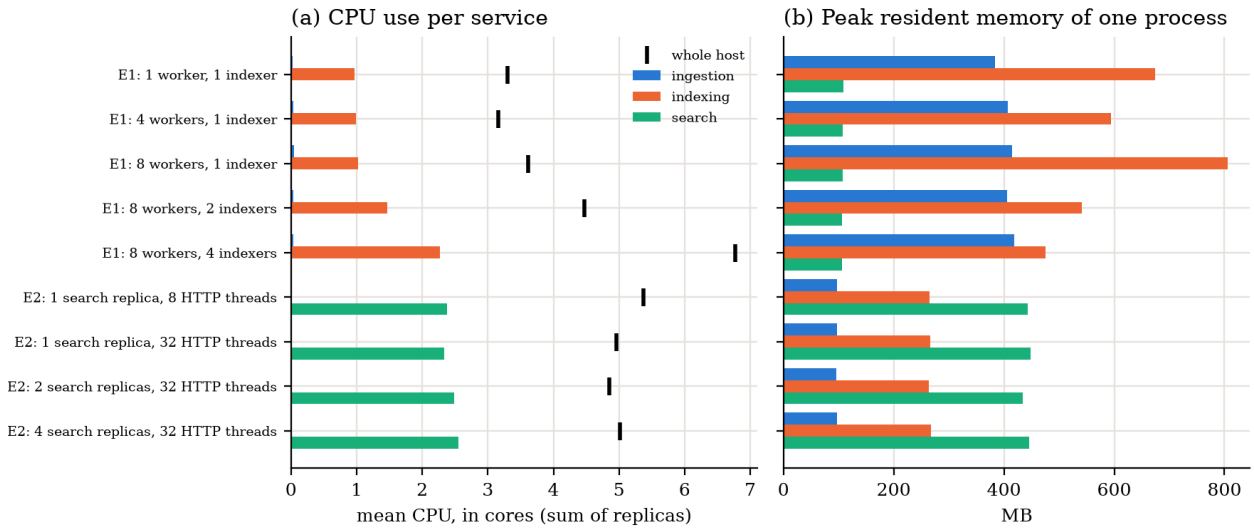


Figure 4. Resource use per configuration. (a) Mean CPU of each service, summed over its replicas, in cores; the black mark is the whole host (28 logical CPUs). (b) Peak resident memory of one process of each service.

- **E1.** Indexing is the only busy service: about one core on average with one replica (peaks of 4–5 cores while several books are tokenized at once) and 1.5–2.3 cores with 2–4 replicas, without any gain in throughput. Ingestion from a local folder uses almost no CPU. The host stays between 3 and 7 cores.
- **E2.** The search processes use about 2.4 cores in total whatever the number of replicas, which means the same amount of work is just split among more processes.
- **Memory.** An indexing process reaches 470–810 MB during E1, more with fewer replicas, probably because one process then buffers the pending terms of more books until they are published. Ingestion stays around 400 MB, and a search process around 110 MB while idle and 430–450 MB under load. Memory is not a limit on this host.

6.5 JMH microbenchmarks

The microbenchmarks measure the core operations without HTTP. `TextProcessingBenchmark` and `IndexPublishBenchmark` use synthetic books with a 50 000-word vocabulary; `SearchBenchmark` runs a fixed workload of 6 queries (with filters) against an index of 100 or 500 synthetic books split into 1 or 10 segments.

Table 11. JMH results, average time per operation (± error at 99.9 %).

Benchmark	Parameter	Time per operation	Benchmark	Parameter	Time per operation
`parseHeader`	10k / 50k / 100k words	1.40 / 1.52 / 1.39 µs	`publishBatch`	1 book	1.48 ± 0.06 s
`splitHeaderBody`	10k / 50k / 100k words	0.20 / 1.01 / 2.06 ms	`publishBatch`	10 books	4.97 ± 0.15 s
`tokenizeAndCount`	10k / 50k / 100k words	0.66 / 3.29 / 6.86 ms	`publishBatch`	50 books	13.0 ± 7.6 s
`searchWorkload`	100 books, 1 / 10 segments	1.185 / 1.182 ms	`searchWorkload`	500 books, 1 / 10 segments	4.97 / 4.90 ms

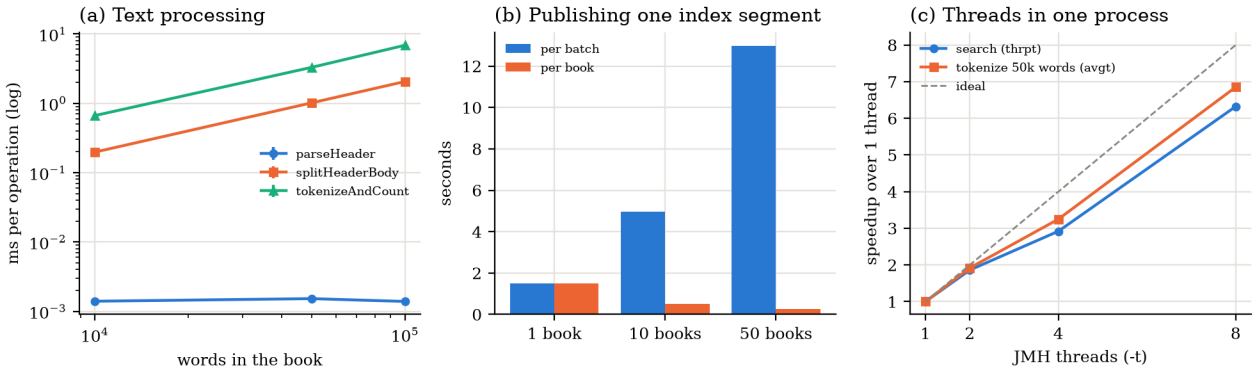


Figure 5. JMH. (a) Text processing time against book size (log–log). (b) Cost of publishing a segment, per batch and per book. (c) Speedup with threads inside one JVM: throughput of the search workload, and books tokenized per unit of time (computed from the average time per operation and the number of threads).

- **Text processing** is linear in the size of the book. Tokenizing costs about 3 times more than splitting header and body (6.9 against 2.1 ms for 100 000 words), and parsing the header does not depend on the size of the book (1.4 µs). A typical book is processed in a few milliseconds, which matches the 10–20 ms of the update call in E1.
- **Publication** costs 1.48 s for a single book, 0.50 s per book in batches of 10 and 0.26 s per book in batches of 50: larger batches amortise the files of the terms shared by many books, at the price of a longer wait before books become searchable. The large error with 50 books reflects the instability of creating very many files.
- **Search** time grows with the corpus (×4.2 from 100 to 500 books) and does not depend on the number of segments (1 or 10). Merging segments is therefore not needed for query speed at this scale; compaction mainly limits the number of files.
- **Threads.** With 8 threads the search workload runs 6.3 times faster and tokenization 6.9 times faster than with one, close to the ideal 8. Both operations are CPU-bound and share no mutable state, so request threads are an effective way to use the cores of one search or indexing process. The smaller gain at 4 threads (2.9–3.2×) is probably due to the hybrid CPU, which mixes performance and efficiency cores.

6.6 Bottlenecks and limitations

1. **First bottleneck: index publication** (file-system bound). It limits the pipeline to about 2.4 books/s and is not solved by more workers or more indexing replicas on one host.
2. **Search** answered up to 12 000 queries/s with sub-millisecond p95 latency at low load; its own limit was not reached because the load generator saturated first. With a larger corpus a query costs more (JMH), and search will become CPU-bound; then threads and replicas will matter.
3. **Limitations of the evidence.** One host shared by services, monitor and load generator; a small corpus (100 books in the integration tests); 3 repetitions per configuration; Windows with NTFS, where creating small files is particularly expensive. A local speedup measures how the architecture uses one machine; it does not show how it will scale on the Stage 3 cluster.

7 Conclusions and future improvements

7.1 Conclusions

- The Stage 1 data layer now runs as three independent services and an orchestrator that cooperate through documented HTTP and shared-storage contracts. The pipeline is correct under concurrency: every benchmark run published all its books with no duplicates, no failures and no errors, and the API passes all 54 checks.
- Immutable segments with an atomically replaced manifest give search a consistent view without any lock on the read path, and the separation between *update* and *publish* makes the cost of consistency explicit and tunable.
- The measured weak point is the physical layout of the segments: one file per term makes publication take tens of seconds and dominates the end-to-end throughput. It confirms the warning of Stage 1 about the hierarchical layout.
- On one host, adding indexing replicas increases contention instead of throughput, while threads inside one process scale almost linearly for CPU-bound work (6.3–6.9× with 8 threads).

7.2 Evolution towards the Stage 3 cluster

Table 12. Single-host assumptions and the change planned for each one.

Current assumption (one host)	Change for the cluster
`localhost` URLs and a static list of replicas in environment variables	Service discovery (for example the DNS of the container orchestrator) and a load balancer in front of the Search replicas
Datalake and datamarts as local files	Shared storage (distributed file system or object storage), or data exposed by each service through its API
`FileLock` and atomic rename	Distributed coordination: locks with leases, or transactions of a database (for example a conditional update of the manifest version)
SQLite, one local writer	Client-server database (for example PostgreSQL) shared by all indexers and searchers
A single Control Module	Persistent message queue (broker) with acknowledgements and retries, or leader election, so that work survives the loss of a node
Search re-reads a local manifest every 500 ms	A “new version published” event, or polling of the manifest in the shared storage

Distribution of work and requests. Indexing can keep the affinity *id* mod *N* as a partition of the books among nodes, and the index can be partitioned by a hash of the term so that each node owns a fraction of the vocabulary; search requests would be spread by the load balancer. **Consistency and failures.** The manifest version remains the commit point; network timeouts and unavailable nodes are handled with the same idempotent operations and retries, now with leases so that the work of a failed node is reassigned.

Future improvements measured as necessary by this stage:

- Store each segment in a few large files (a sorted term dictionary with the postings in one file, or shards by a hash of the term as recommended in Stage 1) instead of one file per term. This attacks the first bottleneck directly.
- Tune `PUBLISH_BATCH` and the publication interval to balance throughput against the time until a book is searchable.
- Repeat E2 with the load generator on another machine (or a compiled load tool) and a larger corpus, to measure the real limit of the search replicas; repeat the whole benchmark on Linux to separate the effect of NTFS.